Prompt Engineering Class Project Specification

Course: Prompt Engineering for Senior Students

Project: Extending PromptTemplate with Advanced Techniques

Project Overview

In this project, you will extend the `PromptTemplate` class by implementing one or two advanced prompt engineering techniques. You will then demonstrate the effectiveness of your implementation by applying it to a real-world use case and comparing results against a baseline (no technique) approach.

Learning Objectives

By completing this project, you will be able to:

1. Analyze and select appropriate prompt engineering techniques for specific use cases
2. Implement reusable prompt engineering patterns in Python
3. Evaluate the effectiveness of prompt engineering techniques through systematic comparison
4. Communicate technical design decisions and experimental findings clearly

---

Project Requirements

Part 1: Technique Selection and Research (10%)

Select **one or two** prompt engineering techniques to implement. Options include (but are not limited to):

Technique	Description
Feynman Learning	Explain concepts simply, identify knowledge gaps, then deepen understanding
Collaborative Intelligence	Multiple perspectives or roles collaborate to refine a response
Socratic Questioning	Guide reasoning through systematic questioning before answering
Self-Consistency	Generate multiple reasoning paths and

Technique	Description
	synthesize the most robust answer
Chain-of-Thought (CoT)	Explicit step-by-step reasoning before final answer
Metacognitive Prompting	Model assesses its own confidence and identifies limitations
Persona Chaining	Sequential responses from different expert perspectives
Constraint Scaffolding	Progressive revelation of constraints to manage complexity
Devil's Advocate	Model argues against its initial position to stress-test reasoning
Structured Decomposition	Break complex problems into sub-problems, solve individually, then integrate

Deliverable: A brief written summary (300-500 words) explaining: - Which technique(s) you selected and why - How the technique works conceptually - What types of problems this technique is well-suited for

---

Part 2: Implementation (40%)

Extend the base `PromptTemplate` class to incorporate your selected technique(s).

Base Class Reference

```python
class PromptTemplate:
    """Base prompt template class."""

    def __init__(self, template: str, **kwargs):
        self.template = template
        self.default_params = kwargs

    def format(self, **kwargs) -> str:
        """Format the template with provided parameters."""
        params = {**self.default_params, **kwargs}
        return self.template.format(**params)

    def with_role(self, role: str) -> 'PromptTemplate':
        """Add a role/persona to the prompt."""
        new_template = f"You are {role}.\n\n{self.template}"
        return PromptTemplate(new_template, **self.default_params)

    def with_output_format(self, format_spec: str) -> 'PromptTemplate':
        """Specify the desired output format."""
```

```
        new_template = f"{self.template}\n\nProvide your response in the foll
owing format:\n{format_spec}"
        return PromptTemplate(new_template, **self.default_params)
```

Implementation Requirements

Your extended class should:

1. **Inherit from or compose with** the base `PromptTemplate` class
2. **Implement at least one new method** that applies your chosen technique
3. **Be reusable** across different base prompts and use cases
4. **Include docstrings** explaining the method's purpose and parameters
5. **Handle edge cases** gracefully (e.g., empty inputs, missing parameters)

Example Structure (Feynman Technique)

```
class FeynmanPromptTemplate(PromptTemplate):
    """Prompt template that applies the Feynman learning technique."""

    def with_feynman_technique(self, audience: str = "a beginner with no back
ground") -> 'FeynmanPromptTemplate':
        """
        Apply the Feynman technique: explain simply, identify gaps, then deep
en.

        Args:
            audience: Description of the target audience for the explanation

        Returns:
            A new FeynmanPromptTemplate with the technique applied
        """
        feynman_wrapper = f"""
{self.template}

Apply the Feynman Learning Technique to your response:

Step 1 - Simple Explanation:
Explain the concept as if teaching {audience}. Use simple language, analogies,
 and concrete examples. Avoid jargon.

Step 2 - Identify Gaps:
Review your explanation. Identify any areas where the explanation feels weak,
 hand-wavy, or where you relied on jargon without truly explaining.

Step 3 - Refine and Deepen:
For each gap identified, provide a clearer, more thorough explanation. Connec
t ideas to foundational concepts.

Step 4 - Final Synthesis:
```

```
Provide a polished final explanation that incorporates your refinements.

Structure your response with clear headers for each step.
"""
        return FeynmanPromptTemplate(feynman_wrapper, **self.default_params)
```

Deliverable: - Python source code file(s) containing your implementation - Code should be well-documented and follow PEP 8 style guidelines

---

Part 3: Use Case Application (30%)

Select a use case and demonstrate your technique's effectiveness.

Suggested Use Cases

Category	Example Use Cases
Education	Tutoring, concept explanation, study guide generation
Software Development	Code review, debugging assistance, documentation writing
Research	Literature summarization, hypothesis generation, methodology critique
Business	Report writing, decision analysis, strategy evaluation
Creative	Story development, argument construction, design feedback
Analysis	Data interpretation, trend analysis, risk assessment

Experimental Design

You must run a systematic comparison:

1. **Baseline Condition:** Use the basic `PromptTemplate` (with role and format if desired, but without your new technique)
2. **Treatment Condition:** Use your extended template with the technique applied
3. **Control Variables:**
    - Use the same LLM and parameters (temperature, max tokens, etc.)
    - Use the same base task/question
    - Run multiple trials if possible (recommended: 3-5 per condition)
4. **Documentation:** Record all prompts used and complete responses received

Evaluation Criteria

Evaluate the outputs using relevant criteria for your use case. Consider:

Dimension	Questions to Consider
Accuracy	Is the information correct? Are there factual errors?
Completeness	Does it address all aspects of the task?
Clarity	Is the response easy to understand? Well-organized?
Depth	Does it go beyond surface-level treatment?
Reasoning Quality	Is the logic sound? Are conclusions well-supported?
Usefulness	Would this response actually help the intended user?

Deliverable: Jupyter notebook containing: - All prompts used (baseline and treatment) - Complete LLM responses for each trial - Side-by-side comparison analysis - Quantitative metrics if applicable (e.g., rubric scores, word counts, error counts)

Part 4: Analysis and Reflection (20%)

Write a report analyzing your findings and reflecting on the project.

Required Sections

1. Executive Summary (100-150 words) - Brief overview of technique, use case, and key findings

2. Methodology (200-300 words) - Detailed description of your experimental setup - Justification for your evaluation criteria - Any limitations of your approach

3. Results (300-500 words) - Presentation of findings with specific examples - Quantitative comparison if applicable - Highlight both successes and failures of the technique

4. Discussion (300-500 words) - Why did the technique help (or not help)? - For what types of tasks would this technique be most valuable? - What are the tradeoffs (e.g., token usage, latency, complexity)?

5. Conclusion and Future Work (150-200 words) - Key takeaways - How you would improve the implementation given more time - Other techniques that might complement yours

Deliverable: Written report (PDF or Markdown), 1200-1800 words total

---

Submission Requirements

Submit the following files:

File	Description
`technique_research.md`	Part 1: Technique selection and research summary

File	Description
`prompt_template_extended.py`	Part 2: Your implementation code
`experiment_notebook.ipynb`	Part 3: Jupyter notebook with use case demonstration
`analysis_report.pdf`	Part 4: Analysis and reflection report
`README.md`	Brief overview and instructions for running your code

Timeline

Milestone	Due Date	Deliverable
Technique Selection	Week 1	Submit technique choice for approval
Implementation Draft	Week 2	Working code (may be incomplete)
Experiment Complete	Week 3	Notebook with all trials run
Final Submission	Week 4	All deliverables

Academic Integrity

- You may use LLMs to help write and debug your code, but you must understand and be able to explain every line
- All experimental results must be genuine (no fabricated outputs)
- Cite any external resources, tutorials, or papers that influenced your approach

---

Getting Help

- Office hours: [Insert times]
- Discussion forum: [Insert link]
- For technique clarification, consult the provided readings and ask in class

---

Appendix A: Example Technique Implementations

Collaborative Intelligence Example

```
class CollaborativePromptTemplate(PromptTemplate):
    """Prompt template using collaborative intelligence with multiple perspec
tives."""

    def with_collaborative_perspectives(
        self,
        perspectives: list[dict],
        synthesis_instruction: str = "Synthesize the perspectives into a bala
```

```
nced final answer"
    ) -> 'CollaborativePromptTemplate':
        """
        Apply collaborative intelligence by having multiple perspectives cont
ribute.

        Args:
            perspectives: List of dicts with 'role' and 'focus' keys
                Example: [
                    {"role": "Optimist", "focus": "benefits and opportunities
"},
                    {"role": "Skeptic", "focus": "risks and limitations"},
                    {"role": "Pragmatist", "focus": "practical implementation
"}
                ]
            synthesis_instruction: How to combine the perspectives

        Returns:
            A new CollaborativePromptTemplate with the technique applied
        """
        perspective_sections = []
        for i, p in enumerate(perspectives, 1):
            perspective_sections.append(
                f"Perspective {i} - {p['role']}:\n"
                f"Analyze the problem focusing on {p['focus']}."
            )

        perspectives_text = "\n\n".join(perspective_sections)

        collaborative_wrapper = f"""
{self.template}

Approach this problem using collaborative intelligence with multiple perspect
ives:

{perspectives_text}

Final Synthesis:
{synthesis_instruction}

Structure your response with clear sections for each perspective, followed by
 the synthesis.
"""
        return CollaborativePromptTemplate(collaborative_wrapper, **self.defa
ult_params)
```

Socratic Questioning Example

```python
class SocraticPromptTemplate(PromptTemplate):
    """Prompt template that applies Socratic questioning methodology."""

    def with_socratic_method(
        self,
        question_depth: int = 3
    ) -> 'SocraticPromptTemplate':
        """
        Apply Socratic questioning before providing an answer.

        Args:
            question_depth: Number of clarifying question rounds to simulate

        Returns:
            A new SocraticPromptTemplate with the technique applied
        """
        socratic_wrapper = f"""
{self.template}

Before providing your final answer, apply the Socratic method:

Step 1 - Clarifying Questions:
List {question_depth} questions you would ask to better understand the proble
m. Then answer these questions based on reasonable assumptions or available c
ontext.

Step 2 - Assumption Examination:
What assumptions are embedded in the question or your initial thinking? Chall
enge at least 2 assumptions.

Step 3 - Evidence and Reasoning:
What evidence or reasoning supports different possible answers? Consider mult
iple viewpoints.

Step 4 - Implications:
What are the implications of different answers? What follows if each possibil
ity is true?

Step 5 - Final Answer:
Based on your Socratic exploration, provide a well-reasoned final answer.

Structure your response with clear headers for each step.
"""
        return SocraticPromptTemplate(socratic_wrapper, **self.default_params)
```